

```

import { useState, useRef, useEffect, useMemo } from "react";
import { Search, Send, X, RefreshCw, Settings, Zap, Target, TrendingUp, ChevronDo

/* — THEME — */
const C = {
  bg:"#070d17",surface:"#0d1625",card:"#111e2e",border:"#1a2a3f",borderHover:"#2a
primary:"#00d4ff",primaryDim:"#002a3d",success:"#00e676",successDim:"#003320",
warn:"#ffab40",warnDim:"#3d2800",danger:"#ff4444",dangerDim:"#3d0000",
violet:"#c084fc",violetDim:"#1a0a35",gold:"#ffd700",goldDim:"#2a2000",
text:"#c8d8e8",textDim:"#4a6a8a",textMid:"#8aa0b8",
mono:"'JetBrains Mono','Fira Code',monospace",
};

/* — DEFAULT SETTINGS — */
const DEFAULT_SETTINGS = {
  bankroll: 500, unitSize: 25, maxUnitsPerEntry: 1, kellyFraction: 0.25,
  hitRates: { goblinOver:0.65, standardHigh:0.58, standardMid:0.52, standardLow:0
edgeWeights: { marketGap:60, lineType:25, yourProjection:15 },
  powerMult: { 2:3, 3:6, 4:10, 5:20, 6:40 },
  flexPay: { 2:{"2/2":3}, 3:{"3/3":5,"2/3":1.25}, 4:{"4/4":10,"3/4":2.5}, 5:{"5/5
minEdgeToPlay: 4, excludeDemons: false, excludeSports: [],
  aiModel: "claude-sonnet-4-20250514",
};

/* — PROBABILITY ENGINE — */
function getPickProb(prop, direction, settings) {
  const { hitRates } = settings;
  if (prop.type === "goblin") return direction === "more" ? hitRates.goblinOver :
  if (prop.type === "demon") return direction === "under" ? hitRates.demonUnder
  // Standard line – use market edge if available
  if (prop.trueEdge !== null) {
    const e = direction === "more" ? prop.trueEdge : -prop.trueEdge;
    if (e >= 8) return hitRates.standardHigh;
    if (e >= 3) return hitRates.standardMid;
    return hitRates.standardLow;
  }
  return hitRates.standardMid;
}

function calcPowerEV(picks, directions, stake, settings) {
  if (picks.length < 2) return null;
  const n = picks.length;
  const mult = settings.powerMult[n];
  if (!mult) return null;

```

```

    const probs = picks.map(p => getPickProb(p, directions[p.id] || "more", setting
    const pWin = probs.reduce((a, b) => a * b, 1);
    const ev = pWin * stake * mult - stake;
    const roi = (ev / stake) * 100;
    return { pWin, ev, roi, payout: stake * mult, probs, mult };
}

```

```

function calcFlexEV(picks, directions, stake, settings) {
    if (picks.length < 2) return null;
    const n = picks.length;
    const payouts = settings.flexPay[n];
    if (!payouts) return null;
    const probs = picks.map(p => getPickProb(p, directions[p.id] || "more", setting
    // Binomial distribution assuming independence
    const binom = (n, k) => { let r=1; for(let i=0;i<k;i++) r=r*(n-i)/(i+1); return
    const avgP = probs.reduce((a,b)=>a+b,0)/n;
    let totalEV = -stake;
    const scenarios = [];
    for (const [key, mult] of Object.entries(payouts)) {
        const k = parseInt(key.split("/") [0]);
        const p = binom(n, k) * Math.pow(avgP, k) * Math.pow(1-avgP, n-k);
        const payout = stake * mult;
        totalEV += p * payout;
        scenarios.push({ key, mult, p, payout, contribution: p * payout });
    }
    const roi = (totalEV / stake) * 100;
    return { ev: totalEV, roi, scenarios, probs };
}

```

```

function kellyStake(pWin, mult, bankroll, fraction) {
    const b = mult - 1;
    const q = 1 - pWin;
    const kelly = (b * pWin - q) / b;
    const recommended = Math.max(0, kelly * fraction * bankroll);
    return { kelly, recommended };
}

```

/* — OPTIMIZER ENGINE — */

```

function combinations(arr, k) {
    if (k === 0) return [[]];
    if (arr.length < k) return [];
    const [first, ...rest] = arr;
    const withFirst = combinations(rest, k-1).map(c => [first, ...c]);
    const withoutFirst = combinations(rest, k);
    return [...withFirst, ...withoutFirst];
}

```

```

function getPropStrategy(prop, strategy) {
  if (strategy === "goblinHunter") return prop.type === "goblin" ? "more" : null
  if (strategy === "demonUnder") return prop.type === "demon" ? "under" : null
  if (strategy === "balanced") return prop.type !== "demon" ? "more" : null
  if (strategy === "maxEV") return prop.trueEdge > 0 ? "more" : prop.true
  return "more";
}

function runOptimizer({ props, strategy, pickCount, entryType, stake, settings })
  const dirs = {};
  const eligible = props.filter(p => {
    const dir = getPropStrategy(p, strategy);
    if (!dir) return false;
    if (strategy !== "demonUnder" && p.type === "demon") return false;
    dirs[p.id] = dir;
    return true;
  });

  // Score each prop individually
  const scored = eligible.map(p => {
    const dir = dirs[p.id];
    const prob = getPickProb(p, dir, settings);
    return { ...p, dir, prob, logProb: Math.log(prob) };
  }).sort((a,b) => b.logProb - a.logProb);

  // Take top N for combination search (limit for performance)
  const pool = scored.slice(0, Math.min(25, scored.length));
  const combos = combinations(pool, pickCount);

  const results = combos.map(combo => {
    const comboProbs = combo.map(p => p.prob);
    const pWin = comboProbs.reduce((a,b)=>a*b,1);
    let ev, roi, payout;
    if (entryType === "power") {
      const mult = settings.powerMult[pickCount] || 0;
      payout = stake * mult;
      ev = pWin * payout - stake;
      roi = (ev / stake) * 100;
    } else {
      const fp = settings.flexPay[pickCount] || {};
      const binom = (n,k)=>{let r=1;for(let i=0;i<k;i++)r=r*(n-i)/(i+1);return r;
      const avgP = comboProbs.reduce((a,b)=>a+b,0)/pickCount;
      ev = -stake;
      for (const [key, mult] of Object.entries(fp)) {
        const k = parseInt(key.split("/")[0]);
        const p = binom(pickCount,k)*Math.pow(avgP,k)*Math.pow(1-avgP,pickCount-k
        ev += p * stake * mult;

```

```

    }
    roi = (ev / stake) * 100;
    payout = null;
  }
  // Diversity bonus: penalize teammate stacks
  const teamCounts = combo.reduce((a,p)=>{a[p.team]=(a[p.team]||0)+1;return a;})
  const stackPenalty = Object.values(teamCounts).filter(c=>c>=2).length * 0.02;
  const adjustedEV = ev * (1 - stackPenalty);
  return { picks: combo, dirs: Object.fromEntries(combo.map(p=>[p.id,p.dir])),
  }).sort((a,b) => b.ev - a.ev).slice(0, 10);

  return results;
}

/* — PP API — */
async function fetchPPSlate(leagueId=null){
  const p=new URLSearchParams({per_page:"500",single_stat:"true",include:"new_pla
  if(leagueId) p.set("league_id",leagueId);
  const r=await fetch(`https://api.prizepicks.com/projections?${p}`, {headers:{"Ac
  if(!r.ok) throw new Error(`PP API ${r.status}`);
  return r.json();
}
async function fetchPPLeagues(){
  const r=await fetch("https://api.prizepicks.com/leagues",{headers:{"Accept":"ap
  if(!r.ok) throw new Error(`PP API ${r.status}`);
  return r.json();
}
function parsePP(d){
  const pm={},lm={};
  for(const i of(d.included||[])){
    if(i.type==="new_player") pm[i.id]=i.attributes;
    if(i.type==="league") lm[i.id]=i.attributes;
  }
  return(d.data||[]).map(proj=>{
    const pId=proj.relationships?.new_player?.data?.id;
    const lId=proj.relationships?.league?.data?.id;
    const p=pm[pId]||{},l=lm[lId]||{};
    const line=parseFloat(proj.attributes.line_score)||0;
    const type=(proj.attributes.line_type||"standard").toLowerCase();
    return{id:proj.id,player:p.name||"Unknown",team:p.team||"-",sport:l.name||p.s
      leagueId:lId,stat:proj.attributes.stat_type||"-",line,line,type,
      description:proj.attributes.description||"",startTime:proj.attributes.start
      marketLine:null,trueEdge:null,marketDk:null,marketFd:null};
  });
}
// Simulates Odds API response — replaced by real backend call in Replit
const MOCK_MARKET={

```

```

    "Anthony Davis":{dk:28.5,fd:28.0,espn:28.5},"LeBron James":{dk:7.5,fd:7.5,espn:
    "Shai Gilgeous-Alexander":{dk:33.5,fd:33.0,espn:33.5},"Chet Holmgren":{dk:29.5,
    "Pascal Siakam":{dk:6.5,fd:6.5,espn:7.0},"Austin Reaves":{dk:1.5,fd:1.5,espn:1.
    "Shohei Ohtani":{dk:2.0,fd:2.0,espn:1.5},"Aaron Judge":{dk:1.5,fd:1.5,espn:1.5}
    "Connor McDavid":{dk:0.5,fd:0.5,espn:0.5},"A'ja Wilson":{dk:24.5,fd:24.0,espn:2
    "Jalen Brunson":{dk:29.5,fd:30.0,espn:29.5},
  };
function injectMarket(props){
  return props.map(p=>{
    const m=MOCK_MARKET[p.player];
    if(!m) return p;
    const ml=+(m.dk+m.fd+m.espn)/3).toFixed(2);
    const gap=+(p.line-ml).toFixed(2);
    return {...p,marketLine:ml,marketDk:m.dk,marketFd:m.fd,trueEdge:+(-gap/ml*100)
  });
}

```

/* — CLAUDE — */

```

async function callClaude(messages,ctx,mode="analyst"){
  const weaponPrompts={
    analyst:`You are a professional prop trading analyst — not a chatbot, not a h

```

LIVE SLATE WITH MARKET COMPARISON:

\${ctx}

OPERATING RULES:

1. Lead with the single most actionable finding. Never bury the lead.
2. Goblin OVER $\approx$ 65% hit prob. Demon UNDER $\approx$ 65%. Demon OVER $\approx$ 35% — never recomm
3. True Edge = $(\text{MarketLine} - \text{PPLine}) / \text{MarketLine} \times 100$. Positive = OVER value.
4. Correlation kills power plays. NBA teammates share possessions. Flag it immedi
5. GTD players in power plays = entry-destroying risk. Always mention injury stat
6. If the market moved against a pick after opening, sharps may disagree. Say so.
7. EV is the only metric that matters long-term. Mention it in every lineup evalu
8. Be honest about uncertainty. If you don't have data to support a claim, say so
9. Never tell the user what they want to hear. Tell them what the data says.
10. Short answers beat long answers. A sharp insight in 2 sentences > a paragraph

anomaly:`You are scanning today's PrizePicks slate for market anomalies — sit

FULL SLATE:

\${ctx}

YOUR TASK:

1. Identify props where True Edge is unusually high (>10%) and explain the probab
2. Flag any picks that look like traps — high edge on paper but underlying risk
3. Note any significant line movements and what they likely mean
4. Surface any hidden correlations in today's highest-edge plays

5. Give a 3-sentence summary of today's slate quality overall

Be specific. Name players and numbers. No generic advice.`,

contradiction:`You are a risk analyst checking for contradictions and hidden

FULL SLATE:

\${ctx}

SCAN FOR:

1. High-edge picks involving players with injury designations
2. Teammate stacks in the top plays (correlation risk)
3. Picks where the line type (demon/goblin) contradicts the recommended direction
4. Any props where multiple signals are pointing in opposite directions
5. Situations where the "right" play mathematically is the emotionally uncomforta

Format as a numbered list. Each item: [RISK TYPE] - Player/Pick - Explanation - R

stresstest:`You are stress-testing a player's entry before they lock it in.

FULL SLATE:

\${ctx}

YOUR JOB:

1. Identify the single most likely pick to fail and exactly why
2. Identify any hidden correlation between picks in this entry
3. Calculate what the entry's true win probability is (accounting for correlation
4. Recommend whether to submit as-is, swap one pick, or rebuild
5. If you'd rebuild, state exactly which pick to replace and with what

Be direct. This person is about to put money down. Don't soften anything.`,

};

```
const r=await fetch("https://api.anthropic.com/v1/messages",{
  method:"POST",headers:{"Content-Type":"application/json"},
  body:JSON.stringify({
    model:"claude-sonnet-4-20250514",max_tokens:1200,
    system:weaponPrompts[mode]||weaponPrompts.analyst,
    messages
  }),
});
const d=await r.json();
return d.content?.map(b=>b.text||"").join("")||"No response.";
}
```

/* — SHARED UI — */

```
function Pill({children,color="dim",size="sm"}){
```

```

const cols={primary:{bg:C.primaryDim,fg:C.primary,bd:C.primary+"44"},success:{b
const c=cols[color]||cols.dim;
return<span style={{background:c.bg,color:c.fg,border:`1px solid ${c.bd}`,paddi
}
function LinePill({type}){if(type==="demon")return<Pill color="danger">👿 DEMON</
function Card({children,style={}}){return<div style={{background:C.card,border:`1
function SHead({icon,title,right}){return<div style={{display:"flex",justifyConte
function Spinner(){return<div style={{display:"flex",alignItems:"center",justifyC
function StatBox({label,value,color=C.primary,sub}){return<div style={{background
function Slider({label,value,min,max,step=1,onChange,format=v=>v}){return<div sty

/* ===== PAGES ===== */

/* — SLATE BOARD — */
function SlateBoard({props,loading,error,picks,onToggle,projections,onProjection,
const[search,setSearch]=useState("");
const[lineType,setLineType]=useState("all");
const[statFilter,setStatFilter]=useState("all");
const[sortCol,setSortCol]=useState("trueEdge");
const[sortDir,setSortDir]=useState(-1);
const statTypes=["all",...new Set(props.map(p=>p.stat))].sort();
const filtered=props
  .filter(p=>lineType==="all"||p.type===lineType)
  .filter(p=>statFilter==="all"||p.stat===statFilter)
  .filter(p=>!search||p.player.toLowerCase().includes(search.toLowerCase())||p.
  .sort((a,b)=>{let av=a[sortCol],bv=b[sortCol];if(typeof av==="string"){av=av.
function ts(col){if(sortCol===col)setSortDir(d=>-d);else{setSortCol(col);setSor
const TH=({col,label,right=false})=><th onClick={()=>ts(col)} style={{padding:"
const TD=({children,style={}})=><td style={{padding:"9px 11px",borderBottom:`1p

return(
  <div style={{display:"flex",flexDirection:"column",height:"100%",padding:20,g
    <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
      <div><h1 style={{margin:0,fontSize:22,fontWeight:700,color:C.text}}>Live
      <button onClick={onRefresh} style={{display:"flex",alignItems:"center",ga
    </div>
    <div style={{display:"flex",gap:5,flexWrap:"wrap"}}>
      <button onClick={()=>onLeague(null)} style={{padding:"5px 12px",borderRad
        {leagues.map(l=><button key={l.id} onClick={()=>onLeague(l.id)} style={{p
    </div>
    <div style={{display:"flex",gap:8,flexWrap:"wrap",alignItems:"center"}}>
      <div style={{position:"relative"}}><Search style={{position:"absolute",le
      <div style={{display:"flex",gap:4}}>
        {["all","standard","goblin","demon"].map(t=><button key={t} onClick={()
      </div>
      <select value={statFilter} onChange={e=>setStatFilter(e.target.value)} st
        {statTypes.map(s=><option key={s} value={s}>{s==="all"? "All Stats":s}</

```

```

    </select>
  </div>
  <div style={{flex:1,background:C.card,border:`1px solid ${C.border}`,border
    {loading?<Spinner/>:error?(
      <div style={{padding:40,textAlign:"center"}}><div style={{fontSize:28,m
    ):(
      <table style={{width:"100%",borderCollapse:"collapse"}}>
        <thead><tr>
          <TH col="player" label="PLAYER"/><TH col="team" label="TEAM"/><TH c
          <TH col="stat" label="STAT"/><TH col="line" label="PP LINE" right/>
          <TH col="marketLine" label="MKT AVG" right/><TH col="trueEdge" labe
          <th style={{padding:"9px 11px",fontFamily:C.mono,fontSize:10,color:
          <th style={{padding:"9px 11px",fontFamily:C.mono,fontSize:10,color:
          <th style={{padding:"9px 11px",background:C.surface,borderBottom:`1
        </tr></thead>
        <tbody>
          {filtered.length===0&&<tr><td colSpan={11} style={{padding:40,textA
          {filtered.map(p=>{
            const inP=picks.some(x=>x.id===p.id);
            const dir=directions[p.id]||"more";
            const prob=getPickProb(p,dir,settings);
            const hasM=p.marketLine!==null;
            const truePos=hasM&&p.trueEdge>0;
            const big=hasM&&Math.abs(p.trueEdge)>=5;
            const rowBg=inP?"#001a0a":p.type==="demon"?"#1a0005":p.type==="go
            return(
              <tr key={p.id} style={{background:rowBg}}>
                <TD><div style={{fontWeight:600,color:C.text,fontSize:13}}>{p
                <TD><span style={{fontFamily:C.mono,fontSize:12,color:C.textM
                <TD><Pill color="dim" size="xs">{p.sport}</Pill></TD>
                <TD><span style={{fontFamily:C.mono,fontSize:12}}>{p.stat}</s
                <TD style={{textAlign:"right"}}><span style={{fontFamily:C.mo
                <TD><LinePill type={p.type}/></TD>
                <TD style={{textAlign:"right"}}>{hasM?<div style={{display:"f
                <TD style={{textAlign:"right"}}>{hasM?<span style={{fontFamil
                <TD style={{textAlign:"right"}}><span style={{fontFamily:C.mo
                <TD style={{textAlign:"right"}}><input value={projections[p.i
                <TD><button onClick={()=>onToggle(p)} disabled={!inP&&picks.l
              </tr>
            );
          }}}
        </tbody>
      </table>
    )}
  </div>
</div>
);

```



```

}

/* — OPTIMIZER — */
const STRATEGIES = [
  { id:"goblinHunter", label:"👹 Goblin Hunter", desc:"Only goblin lines OVER. M",
  { id:"demonUnder", label:"👿 Demon Under", desc:"Demon lines played UNDER.
  { id:"balanced", label:"⚖️ Balanced Value", desc:"Standard + goblin lines w
  { id:"maxEV", label:"⚡ Max EV", desc:"Finds highest expected v
];

function OptimizerPage({ props, onLoadEntry, settings }) {
  const [strategy, setStrategy] = useState("goblinHunter");
  const [pickCount, setPickCount] = useState(4);
  const [entryType, setEntryType] = useState("power");
  const [stake, setStake] = useState(25);
  const [results, setResults] = useState([]);
  const [running, setRunning] = useState(false);
  const [expandedIdx, setExpandedIdx] = useState(null);

  function run() {
    if (props.length === 0) return;
    setRunning(true);
    setResults([]);
    setExpandedIdx(null);
    setTimeout(() => {
      const r = runOptimizer({ props, strategy, pickCount, entryType, stake, sett
      setResults(r);
      setRunning(false);
    }, 80);
  }

  const stratInfo = STRATEGIES.find(s => s.id === strategy);
  const evColor = ev => ev > 0 ? C.success : ev > -stake*0.2 ? C.warn : C.danger;

  return (
    <div style={{ padding:20, display:"flex", flexDirection:"column", gap:16, hei
      <div>
        <h1 style={{ margin:0, fontSize:22, fontWeight:700, color:C.text }}>Lineu
        <div style={{ fontSize:12, color:C.textDim, marginTop:4 }}>Finds the high
      </div>

      { /* Strategy selector */ }
      <Card>
        <SHead icon="🎯" title="Strategy"/>
        <div style={{ padding:14, display:"grid", gridTemplateColumns:"repeat(2,1
          {STRATEGIES.map(s => (
            <button key={s.id} onClick={() => setStrategy(s.id)} style={{ padding

```

```

        <div style={{ fontWeight:700, fontSize:13, color:strategy===s.id?C[
        <div style={{ fontSize:11, color:C.textDim }}>{s.desc}</div>
        </button>
    )}}
</div>
</Card>

{/* Config row */}
<div style={{ display:"grid", gridTemplateColumns:"repeat(3,1fr)", gap:12 }
    <Card style={{ padding:16 }}>
        <div style={{ fontSize:11, fontFamily:C.mono, color:C.textDim, marginBo
        <div style={{ display:"flex", gap:6 }}>
            {[2,3,4,5,6].map(n => <button key={n} onClick={()=>setPickCount(n)} s
        </div>
    </Card>
    <Card style={{ padding:16 }}>
        <div style={{ fontSize:11, fontFamily:C.mono, color:C.textDim, marginBo
        <div style={{ display:"flex", gap:6 }}>
            [{"power","flex"].map(t => <button key={t} onClick={()=>setEntryType(
        </div>
    </Card>
    <Card style={{ padding:16 }}>
        <div style={{ fontSize:11, fontFamily:C.mono, color:C.textDim, marginBo
        <div style={{ display:"flex", gap:5 }}>
            {[10,25,50,100].map(a => <button key={a} onClick={()=>setStake(a)} st
        </div>
    </Card>
</div>

{/* Hit rate reference */}
<div style={{ padding:"10px 14px", background:"#0a1525", border:`1px solid
    <div style={{ fontSize:11, color:C.textDim }}>Hit rate assumptions from S
    {[["👹 Goblin OVER", (settings.hitRates.goblinOver*100).toFixed(0)+"%", "su
        <div key={1} style={{ fontSize:11, fontFamily:C.mono }}><span style={{
        )}}
    </div>

<button onClick={run} disabled={running||props.length===0} style={{ padding
    {running?"⌚ RUNNING OPTIMIZER...":`⚡ RUN OPTIMIZER - ${stratInfo?.label}`
</button>

{/* Results */}
{results.length > 0 && (
    <div style={{ display:"flex", flexDirection:"column", gap:10 }}>
        <div style={{ fontSize:12, fontFamily:C.mono, color:C.textDim, letterSp
        {results.map((r, idx) => {
            const isExp = expandedIdx === idx;

```

```

const kelly = entryType==="power" ? kellyStake(r.pWin, settings.power
return (
  <div key={idx} style={{ background:C.card, border:`1px solid ${idx=
    <div onClick={() => setExpandedIdx(isExp?null:idx)} style={{ padd
      <div style={{ fontFamily:C.mono, fontSize:18, fontWeight:700, c
    <div style={{ flex:1 }}>
      <div style={{ display:"flex", gap:8, flexWrap:"wrap", marginB
        {r.picks.map(p => (
          <div key={p.id} style={{ padding:"3px 8px", background:p.
            <span style={{ fontSize:11, fontWeight:600, color:C.tex
            <span style={{ fontSize:10, color:C.textDim, fontFamily
            <span style={{ fontSize:10, fontFamily:C.mono, color:p.
          </div>
        )})}
      </div>
    </div>
    <div style={{ display:"flex", gap:16 }}>
      <span style={{ fontSize:11, fontFamily:C.mono, color:C.text
        {r.payout&&<span style={{ fontSize:11, fontFamily:C.mono, c
        {kelly&&kelly.recommended>0&&<span style={{ fontSize:11, fo
      </div>
    </div>
    <div style={{ textAlign:"right", minWidth:90 }}>
      <div style={{ fontFamily:C.mono, fontSize:20, fontWeight:700,
        <div style={{ fontSize:10, color:C.textDim, fontFamily:C.mono
          <div style={{ fontSize:11, fontFamily:C.mono, color:evColor(r
        </div>
        <div style={{ color:C.textDim }}>{isExp?<ChevronDown style={{wi
      </div>
    {isExp && (
      <div style={{ borderTop:`1px solid ${C.border}`, padding:14, ba
        <div style={{ display:"grid", gridTemplateColumns:"repeat(4,1
          <StatBox label="WIN PROBABILITY" value={r.pWin?(r.pWin*100)
          <StatBox label="EXPECTED VALUE" value={ (r.ev>=0?"+"+"")+r.e
          <StatBox label="ROI" value={ (r.roi>=0?"+"+"")+r.roi.toFixed
            {kelly?<StatBox label="KELLY STAKE" value={`$$${kelly.recomm
          </div>
        <div style={{ marginBottom:10 }}>
          <div style={{ fontSize:10, fontFamily:C.mono, color:C.textD
            {r.picks.map((p,i) => {
              const prob = r.probs?.[i] ?? 0.5;
              return(
                <div key={p.id} style={{ display:"flex", justifyContent
                  <div><span style={{ fontWeight:600, fontSize:13 }}>{p
                  <div style={{ display:"flex", gap:12, alignItems:"cen
                    < Pill color={r.dirs[p.id]=== "under"? "warn": "success
                    < LinePill type={p.type}/>
                    <span style={{ fontFamily:C.mono, fontSize:12, font

```

```

        </div>
      </div>
    );
  }}}
</div>
<button onClick={()=>onLoadEntry(r.picks,r.dirs)} style={{ pa
  → Load into Entry Builder
</button>
</div>
  })
</div>
);
  }}}
</div>
  )}
  {results.length===0&&!running&&props.length===0&&<div style={{ textAlign:"c
</div>
  );
}

```

/* — ENTRY BUILDER — */

```

function EntryBuilder({ picks, onToggle, directions, onDirection, settings }) {
  const [stake, setStake] = useState(settings.unitSize);
  const [mode, setMode] = useState("power");
  const n = picks.length;
  const powerEV = calcPowerEV(picks, directions, stake, settings);
  const flexEV = calcFlexEV(picks, directions, stake, settings);
  const activeEV = mode === "power" ? powerEV : flexEV;
  const teamCounts = picks.reduce((a,p)=>{a[p.team]=(a[p.team]||0)+1;return a;},{
  const stacks = Object.entries(teamCounts).filter(([c])=>c>=2);
  const evColor = ev => ev > 0 ? C.success : ev > -stake*0.2 ? C.warn : C.danger;

  return (
    <div style={{ padding:20, display:"flex", gap:20, height:"100%" }}>
      <div style={{ flex:1, display:"flex", flexDirection:"column", gap:12 }}>
        <div style={{ display:"flex", justifyContent:"space-between", alignItems:
          <h1 style={{ margin:0, fontSize:22, fontWeight:700, color:C.text }}>Ent
          <Pill color={n>=2?"success":"dim"}>{n}/6 picks</Pill>
        </div>
        {stacks.length>0&&<div style={{ padding:"10px 14px", background:"#1a1000"
        <Card style={{ flex:1 }}>
          <SHead icon="🎯" title={`Active Picks (${n}/6)`}/>
          <div style={{ padding:12, display:"flex", flexDirection:"column", gap:8
            {n===0&&<div style={{ textAlign:"center", padding:60, color:C.textDim
              {picks.map(p => {
                const dir = directions[p.id] || "more";
                const prob = getPickProb(p, dir, settings);

```

```

return(
  <div key={p.id} style={{ padding:"12px 14px", background:C.surface
    <div style={{ display:"flex", justifyContent:"space-between", a
      <div>
        <div style={{ fontWeight:700, fontSize:14, color:C.text }}>
        <div style={{ fontSize:11, color:C.textDim, fontFamily:C.mo
          {p.team} · {p.stat} · <span style={{ color:C.primary }}>{
            {p.trueEdge!==null&&<span style={{ marginLeft:8, color:p.
              <span style={{ marginLeft:8, color:prob>=0.6?C.success:pr
            </div>
          </div>
        </div>
      <div style={{ display:"flex", alignItems:"center", gap:8 }}>
        [{"more","less"].map(d=><button key={d} onClick={()=>onDire
        <LinePill type={p.type}/>
        <button onClick={()=>onToggle(p)} style={{ background:C.dan
      </div>
    </div>
  </div>
  );
  }}}
</div>
</Card>
</div>

<div style={{ width:280, display:"flex", flexDirection:"column", gap:12 }}>
  { /* EV Panel */
    {n >= 2 && activeEV && (
      <Card style={{ padding:16 }}>
        <div style={{ fontSize:11, fontFamily:C.mono, color:C.textDim, margin
        <div style={{ display:"grid", gridTemplateColumns:"1fr 1fr", gap:10,
          <div style={{ background:C.surface, borderRadius:6, padding:"10px 1
            <div style={{ fontSize:10, color:C.textDim, fontFamily:C.mono }}>
            <div style={{ fontFamily:C.mono, fontSize:20, fontWeight:700, col
          </div>
          <div style={{ background:C.surface, borderRadius:6, padding:"10px 1
            <div style={{ fontSize:10, color:C.textDim, fontFamily:C.mono }}>
            <div style={{ fontFamily:C.mono, fontSize:20, fontWeight:700, col
          </div>
        </div>
        <div style={{ background:C.surface, borderRadius:6, padding:"10px 12p
          <div style={{ display:"flex", justifyContent:"space-between", fontS
            <span style={{ color:C.textDim }}>ROI</span>
            <span style={{ color:evColor(activeEV.roi), fontWeight:700 }}>{ac
          </div>
        </div>
      </div>
    { /* Probability chain */
    <div style={{ fontSize:10, color:C.textDim, fontFamily:C.mono, margin

```

```

    <div style={{ display:"flex", alignItems:"center", gap:4, flexWrap:"w
      {picks.map((p,i)=>{
        const pr=(activeEV.probs?.[i]?0.5)*100;
        return<span key={p.id} style={{ fontFamily:C.mono, fontSize:11 }}
          <span style={{ color:pr>=60?C.success:pr>=50?C.warn:C.danger }}
            {i<picks.length-1&&<span style={{ color:C.textDim }}> × </span>
          </span>;
        )))
      <span style={{ fontFamily:C.mono, fontSize:11, color:C.textDim }}>
      <span style={{ fontFamily:C.mono, fontSize:11, fontWeight:700, colo
    </div>
    {/* Kelly recommendation */}
    {mode==="power"&&powerEV&&()=>{
      const k=kellyStake(powerEV.pWin,settings.powerMult[n]||1,settings.b
      return k.kelly>0&&<div style={{ marginTop:10, padding:"8px 10px", b
    })() }
  </Card>
)

{/* Payout config */}
<Card style={{ padding:16 }}>
  <div style={{ fontSize:11, fontFamily:C.mono, color:C.textDim, marginBo
  <div style={{ display:"flex", gap:5, marginBottom:8 }}>
    {[10,25,50,100].map(a=><button key={a} onClick={()=>setStake(a)} styl
  </div>
  <input value={stake} onChange={e=>setStake(parseFloat(e.target.value)||
  <div style={{ display:"flex", gap:5, marginBottom:14 }}>
    [{"power","flex"].map(m=><button key={m} onClick={()=>setMode(m)} sty
  </div>
  {n=>2&&mode==="power"&&powerEV&&<div style={{ background:C.surface, bor
  {n=>2&&mode==="flex"&&flexEV&&<div style={{ background:C.surface, borde
  {n<2&&<div style={{ fontSize:12, color:C.textDim, textAlign:"center", p
  </Card>
</div>
</div>
);
}

/* — SETTINGS — */
function SettingsPage({ settings, onSave }) {
  const [s, setS] = useState(settings);
  const set = (path, val) => {
    setS(prev => {
      const next = JSON.parse(JSON.stringify(prev));
      const keys = path.split(".");
      let obj = next;
      for (let i = 0; i < keys.length - 1; i++) obj = obj[keys[i]];

```

```

    obj[keys[keys.length-1]] = val;
    return next;
  });
};

const Section = ({ title, children }) => (
  <Card style={{ marginBottom:16 }}>
    <SHead icon="🔍" title={title}/>
    <div style={{ padding:16 }}>{children}</div>
  </Card>
);

const Row = ({ label, sub, children }) => (
  <div style={{ display:"flex", justifyContent:"space-between", alignItems:"center" }}>
    <div><div style={{ fontSize:13, color:C.text }}>{label}</div>{sub}&&<div style={{ display:"flex", gap:10 }}>
      <div>{children}</div>
    </div>
  </div>
);

const NumInput = ({ val, onChange, prefix="", suffix="" }) => (
  <div style={{ display:"flex", alignItems:"center", gap:4 }}>
    {prefix}&&<span style={{ fontFamily:C.mono, fontSize:12, color:C.textDim }}>
      <input type="number" value={val} onChange={e=>onChange(parseFloat(e.target.value))}>
    </span>{suffix}&&<span style={{ fontFamily:C.mono, fontSize:12, color:C.textDim }}>
    </div>
);

const rateRow = (key, label, color) => (
  <div key={key} style={{ display:"flex", justifyContent:"space-between", alignItems:"center" }}>
    <span style={{ fontSize:13, color:C.text }}>{label}</span>
    <div style={{ display:"flex", alignItems:"center", gap:8 }}>
      <input type="range" min={20} max={85} step={1} value={Math.round(s.hitRate)} />
      <span style={{ fontFamily:C.mono, fontSize:13, fontWeight:700, color:C[color] }}>{s.hitRate}</span>
    </div>
  </div>
);

return (
  <div style={{ padding:20, overflowY:"auto", height:"100%" }}>
    <div style={{ display:"flex", justifyContent:"space-between", alignItems:"center" }}>
      <div><h1 style={{ margin:0, fontSize:22, fontWeight:700, color:C.text }}>Bankroll Management</h1>
      <button onClick={()=>onSave(s)} style={{ padding:"10px 20px", background:C[primary] }}>Save</button>
    </div>

    <Section title="Bankroll Management">
      <Row label="Current Bankroll" sub="Total PP bankroll available"><NumInput val={s.bankroll} />
      <Row label="Unit Size" sub="Default stake per entry"><NumInput val={s.unit} />
    </Section>
  </div>
);

```

```

<Row label="Kelly Fraction" sub="0.25 = quarter Kelly (recommended for PP
  <div style={{ display:"flex", alignItems:"center", gap:8 }}>
    <input type="range" min={5} max={100} step={5} value={Math.round(s.ke
    <span style={{ fontFamily:C.mono, fontSize:13, fontWeight:700, color:
  </div>
</Row>

<Row label="Max Units Per Entry" sub="Hard cap. Prevents overbetting any
<div style={{ padding:"10px 12px", background:C.primaryDim, border:`1px s
  💡 At your current settings: max entry = <strong style={{ color:C.prima
</div>
</Section>

<Section title="Hit Rate Assumptions">
  <div style={{ fontSize:12, color:C.textDim, marginBottom:14 }}>These driv
  {rateRow("goblinOver",    "👹 Goblin Line – OVER",      "success")}
  {rateRow("standardHigh",  "⚡ Standard – High Market Edge (>5%)", "primar
  {rateRow("standardMid",   "📊 Standard – Mid Edge (0-5%)", "textMid")}
  {rateRow("standardLow",   "📉 Standard – Negative Edge",  "warn")}
  {rateRow("demonUnder",    "👿 Demon Line – UNDER",      "warn")}
  {rateRow("demonOver",     "💀 Demon Line – OVER (trap)",  "danger")}
  <div style={{ padding:"10px 12px", background:C.warnDim, border:`1px soli
    ⚠️ These are estimates. As you log results, update these to reflect you
  </div>
</Section>

<Section title="Payout Config">
  <div style={{ fontSize:12, color:C.textDim, marginBottom:14 }}>PrizePicks
  <div style={{ marginBottom:14 }}>
    <div style={{ fontSize:12, fontFamily:C.mono, color:C.textDim, marginBo
    <div style={{ display:"grid", gridTemplateColumns:"repeat(5,1fr)", gap:
      {[2,3,4,5,6].map(n => (
        <div key={n} style={{ textAlign:"center" }}>
          <div style={{ fontSize:11, color:C.textDim, marginBottom:4, fontF
            <input type="number" value={s.powerMult[n]||0} onChange={e=>set(`
              <div style={{ fontSize:10, color:C.textDim, marginTop:3 }}>×</div
            </div>
          )}}
        </div>
      </div>
    </Section>

<Section title="Edge Score Weights">
  <div style={{ fontSize:12, color:C.textDim, marginBottom:14 }}>How the fi
  {[["marketGap", "Market Gap vs DK/FD/ESPN", "primary", "Primary signal when
    <div key={k} style={{ marginBottom:14 }}>
      <div style={{ display:"flex", justifyContent:"space-between", marginB
        <input type="range" min={0} max={100} step={5} value={s.edgeWeights[k

```



```

        </div>
    )})
</Section>

<Section title="Sharp Player Preferences">
  <Row label="Min Edge to Auto-Flag as PLAY" sub="Props below this threshol
  <Row label="Exclude All Demon OVER lines" sub="Removes them from optimize
    <div onClick={()=>set("excludeDemons",!s.excludeDemons)} style={{ width
      <div style={{ position:"absolute", top:2, left:s.excludeDemons?20:2,
    </div>
  </Row>
  <Row label="AI Model" sub="Haiku is 3× cheaper; Sonnet gives better analy
    <select value={s.aiModel} onChange={e=>set("aiModel",e.target.value)} s
      <option value="claude-sonnet-4-20250514">Claude Sonnet (recommended)<
      <option value="claude-haiku-4-5-20251001">Claude Haiku (cheaper, ~$3/
    </select>
  </Row>
</Section>
</div>
);
}

/* — AI ANALYST — */
function AIAnalyt({ picks, props, settings }) {
  const [msgs, setMsgs] = useState([{"role":"assistant",content:`Intelligence brief
  const [input, setInput] = useState("");
  const [loading, setLoading] = useState(false);
  const [activeMode, setActiveMode] = useState("analyst");
  const bottomRef = useRef(null);
  useEffect(() => { bottomRef.current?.scrollIntoView({ behavior:"smooth" }); },

  const slateCtx = props.slice(0, 100).map(p => {
    const mkt = p.marketLine ? ` | Mkt:${p.marketLine} | TrueEdge:${p.trueEdge>0?
    const prob = getPickProb(p,"more",settings);
    return `${p.player}(${p.team}) - ${p.sport} · ${p.stat}: PP ${p.line} [${p.ty
  }).join("\n");

  const MODES = [
    { id:"anomaly",      label:"🔍 Anomaly Hunt",      desc:"Unusually large ed
    { id:"contradiction",label:"⚠️ Contradiction Check", desc:"Risks hiding in y
    { id:"stresstest",   label:"🎯 Stress Test Entry",   desc:"Find the weakest
    { id:"analyst",      label:"💬 Open Analysis",      desc:"Ask anything dire
  ];

  async function send(text, mode) {
    const m = mode || activeMode;
    const msg = text || input.trim();

```

```

if (!msg || loading) return;
setInput("");
let full = msg;
if (picks.length > 0) {
  const ev = calcPowerEV(picks, {}, settings.unitSize, settings);
  full += `\\n\\n[Entry: ${picks.map(p=>`${p.player} ${p.stat} ${p.line} [${p.t
}
}
const newMsgs = [...msgs, { role:"user", content:full }];
setMsgs(newMsgs);
setLoading(true);
try { const reply = await callClaude(newMsgs, slateCtx, m); setMsgs(p=>[...p,
catch { setMsgs(p=>[...p,{role:"assistant",content:"Error connecting to Claud
setLoading(false);
}

return (
  <div style={{ padding:20, display:"flex", flexDirection:"column", height:"100
    <div style={{ display:"flex", justifyContent:"space-between", alignItems:"c
      <div>
        <h1 style={{ margin:0, fontSize:22, fontWeight:700, color:C.text }}>AI
        <div style={{ fontSize:11, color:C.textDim, marginTop:3 }}>Decision int
      </div>
      <div style={{ fontSize:11, fontFamily:C.mono, color:C.textDim, display:"f
        <span style={{ width:7, height:7, borderRadius:"50%", background:C.succ
          {settings.aiModel.includes("sonnet")?"Claude Sonnet":"Claude Haiku"} }
        </div>
      </div>
    </div>
    <div style={{ display:"grid", gridTemplateColumns:"repeat(4,1fr)", gap:8 }}
      {MODES.map(m=>(
        <button key={m.id} onClick={()=>setActiveMode(m.id)} style={{ padding:"
          <div style={{ fontWeight:700, fontSize:12, color:activeMode===m.id?C.
            <div style={{ fontSize:10, color:C.textDim }}>{m.desc}</div>
          </button>
        )})
      </div>
      {activeMode!=="analyst"&&(
        <button onClick={()=>send(`Run ${activeMode} analysis on today's full sla
          {loading?"⌚ ANALYZING...":`▶ RUN ${MODES.find(m=>m.id===activeMode)||1
        </button>
      )}
    <Card style={{ flex:1, display:"flex", flexDirection:"column", overflow:"hi
      <div style={{ flex:1, overflowY:"auto", padding:16, display:"flex", flexD
        {msgs.map((m,i)=>(
          <div key={i} style={{ display:"flex", justifyContent:m.role==="user"?
            <div style={{ maxWidth:"80%", padding:"12px 16px", borderRadius:8,
              {m.role==="assistant"&&<div style={{ fontSize:10, fontFamily:C.mo
                {m.content}

```

```

        </div>
      </div>
    )))
    {loading}&&<div style={{ display:"flex" }}><div style={{ padding:"12px 1
    <div ref={bottomRef}/>
  </div>
  <div style={{ padding:"11px 16px", borderTop:`1px solid ${C.border}`, dis
    <input value={input} onChange={e=>setInput(e.target.value)} onKeyDown={
    <button onClick={()=>send(input.trim())} disabled={loading||!input.trim
  </div>
</Card>
</div>
);
}

/* — MAIN — */
const NAV=[{id:"slate",label:"Live Slate",icon:"📝"},{id:"optimizer",label:"Optim

export default function App() {
  const [page, setPage]      = useState("slate");
  const [picks, setPicks]    = useState([]);
  const [dirs, setDirs]      = useState({});
  const [projs, setProjs]    = useState({});
  const [props, setProps]    = useState([]);
  const [leagues, setLeagues] = useState([]);
  const [selLeague, setSelLeague] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError]    = useState(null);
  const [lastUp, setLastUp]  = useState(null);
  const [settings, setSettings] = useState(DEFAULT_SETTINGS);

  async function loadSlate(lid) {
    setLoading(true); setError(null);
    try {
      const d = await fetchPPSlate(lid);
      const parsed = parsePP(d);
      setProps(injectMarket(parsed));
      setLastUp(new Date().toLocaleTimeString());
    } catch(e) { setError(e.message); }
    setLoading(false);
  }

  async function loadLeagues() {
    try {
      const d = await fetchPPLeagues();
      setLeagues((d.data||[]).filter(l=>l.attributes?.active).map(l=>({id:l.id,na
    } catch {}
  }

```

```

useEffect(() => { loadLeagues(); loadSlate(null); }, []);

function toggleProp(prop) {
  setPicks(prev => {
    if (prev.some(p=>p.id===prop.id)) return prev.filter(p=>p.id!==prop.id);
    if (prev.length >= 6) return prev;
    return [...prev, prop];
  });
  setDirs(prev => ({ ...prev, [prop.id]: prev[prop.id] || "more" }));
}

function loadOptimizerEntry(optPicks, optDirs) {
  setPicks(optPicks);
  setDirs(prev => ({ ...prev, ...optDirs }));
  setPage("builder");
}

return (
  <div style={{ display:"flex", height:"100vh", background:C.bg, color:C.text,
    <style>{@import url('https://fonts.googleapis.com/css2?family=JetBrains+Mo

    /* Sidebar */
    <div style={{ width:210, background:C.surface, borderRight:`1px solid ${C.b
      <div style={{ padding:"18px 16px 14px", borderBottom:`1px solid ${C.borde
        <div style={{ fontSize:17, fontWeight:800, letterSpacing:"-0.5px" }}><s
          <div style={{ fontSize:10, fontFamily:C.mono, color:C.textDim, marginTo
        </div>
      <nav style={{ flex:1, padding:"10px 8px" }}>
        {NAV.map(n=>(
          <button key={n.id} onClick={()=>setPage(n.id)} style={{ width:"100%",
            onMouseEnter={e=>{if(page!==n.id)e.currentTarget.style.background=C.c
            onMouseLeave={e=>{if(page!==n.id)e.currentTarget.style.background="tr
          ><span style={{ fontSize:15 }}>{n.icon}</span>{n.label}{n.id=="build
        )}}
      </nav>
      {picks.length > 0 && (
        <div style={{ margin:"0 8px 10px", padding:12, background:C.card, borde
          <div style={{ fontSize:10, fontFamily:C.mono, color:C.textDim, margin
            {picks.map(p => {
              const prob = getPickProb(p, dirs[p.id]||"more", settings);
              return(
                <div key={p.id} style={{ display:"flex", justifyContent:"space-be
                  <span style={{ color:C.textMid, overflow:"hidden", textOverflow
                    <div style={{ display:"flex", gap:5, alignItems:"center" }}>
                      <span style={{ fontFamily:C.mono, color:C.primary, fontSize:1
                        <span style={{ fontFamily:C.mono, fontSize:10, color:prob>=0.
                    </div>

```

```

        </div>
    );
  }}}
  {(()=>{
    const ev = calcPowerEV(picks, dirs, settings.unitSize, settings);
    return ev && <div style={{ marginTop:8, padding:"6px 8px", backgrou
      <span style={{ fontFamily:C.mono, fontSize:10, color:C.textDim }}
      <span style={{ fontFamily:C.mono, fontSize:10, fontWeight:700, co
    </div>;
  }) ()}
  <button onClick={()=>setPage("builder")} style={{ marginTop:8, width:
</div>
  })
  <div style={{ padding:"10px 16px", borderTop:`1px solid ${C.border}`, fon
    <div style={{ display:"flex", alignItems:"center", gap:6 }}><span style
      <div style={{ marginTop:3 }}>Bankroll: <span style={{ color:C.primary }
    </div>
  </div>

  { /* Main */}
  <div style={{ flex:1, overflowY:"auto", display:"flex", flexDirection:"colu
    {page==="slate"      && <SlateBoard props={props} loading={loading} error=
    {page==="optimizer" && <OptimizerPage props={props} onLoadEntry={loadOpti
    {page==="builder"   && <EntryBuilder picks={picks} onToggle={toggleProp}
    {page==="ai"        && <AIAnalyst picks={picks} props={props} settings={s
    {page==="settings"  && <SettingsPage settings={settings} onSave={s=>{setS
  </div>
</div>
  );
}

```